Workbook Compare

Developer overview: what the app does, why it exists, and how it is built

Written for: developers joining the project who need to understand its purpose, architecture and day-to-day operation.

1. Purpose

Workbook Compare is an internal Flask web app that answers one question before a Tableau workbook change is published: **what will this change affect?**

A developer uploads the proposed (candidate) version of a workbook. The app downloads the version currently live on Tableau Server (the published version), compares the two, and reports what changed. It then identifies the Custom Views built on that workbook, meaning the saved personal views that could break, along with their owners, so those owners can be notified.

The app is **strictly read-only** with respect to Tableau. It never publishes, overwrites or deletes workbooks or Custom Views on the server.

2. Who uses it and when

- **Workbook developers and reviewers**, before publishing a change to a workbook.
- **Site or content owners**, who need to know which Custom View owners to contact.
- **Anyone browsing Custom Views** across the site, using the read-only directory page.

3. How a compare works

- **Sign in.** The user enters the dashboard passcode. Only a hash of it is stored.
- **Pick a workbook.** The workbook list is fetched live from the Tableau site currently selected.
- **Upload the candidate.** A .twb or .twbx file. A .twbx is a zip archive, so the app extracts the .twb from it.
- **Download the published version.** The app pulls the workbook definition from Tableau and parses it.
- **Parse and diff.** Both files are parsed from XML into a normalized structure, then compared.
- **Classify impact.** The workbook is classified overall, then each Custom View is classified against the diff.
- **Resolve owners.** Owner emails are looked up in a local cache to add account numbers.
- **Review and export.** The results page shows the diff and impacts. A PDF report can be generated in memory and downloaded.

4. What the diff covers

The engine (*workbook_compare_engine.py*) extracts and compares the following from the workbook XML:

- Worksheets, dashboards and hidden sheets
- Fields, including calculations, groups, sets and computed sets
- Parameters and their values
- Filters on worksheets and data sources, and quick-filter card visibility
- Marks, encodings and sorts per worksheet

- Dashboard layout zones
- Data connections, including server, port and class
- Actions (filter, highlight, URL and parameter)

Some items are marked best-effort in the code. Computed sets, for example, depend on a class string that has not been verified against a real Tableau export. If that string differs, the item degrades to 'not detected' rather than failing.

5. Architecture

The app is a single Flask process served by Waitress. Code is organized as follows:

File / folder	Responsibility
app.py	App factory. Creates the Flask app, registers blueprints, the health check and the site switcher. Starts the background account-number sync. Serves with Waitress, with a 10 GB request cap for large uploads.
config.py / config.yaml	Non-secret settings: server URL, host, port, site list and default site. Environment variables override them.
db.py	SQLite store under instance\ (cache.db). Holds app_config (encrypted PAT, PAT name, passcode hash) and account_numbers.
crypto.py	Encrypts and decrypts the PAT at rest.
auth.py	Passcode hashing and verification, plus the login_required decorator.
site_context.py	Tracks the currently selected Tableau site for the session.
tableau_client.py	Wrapper around tableauserverclient. Signs in, lists workbooks and Custom Views, downloads definitions, retries transient failures.
workbook_compare_engine.py	Parses .twb XML, computes the diff, and classifies custom-view impact. The core logic, and the largest module.
custom_view_state.py	Filter and parameter state normalization and delta logic. Partly built; see Section 8.
workbook_compare_pdf.py	Builds the PDF report in memory with reportlab. Sends nothing anywhere.
bigquery_sync.py	Pulls the email to account-number map from BigQuery into the local cache.
routes/	HTTP endpoints: setup.py, auth_routes.py, workbook_compare.py, custom_views.py.
templates/ and static/	Jinja2 pages (setup, login, workbook compare, custom views) and CSS/JS.
tests/	pytest suite for the engine, PDF, routes, Tableau client and custom-view state.
startup.ps1 / startup.sh	Launch scripts. Set APP_HOST, APP_PORT and APP_DATA_DIR, then run app.py.

6. Endpoints

Route	Method	Purpose	Auth
/	GET	Redirects to setup, login or the compare page, depending on state	None
/setup	GET, POST	First-run setup: validates the PAT against the server, then stores it and the passcode hash	None
/login, /logout	GET, POST	Passcode login and session clear	None
/workbook-compare	GET	Workbook picker	Passcode

Route	Method	Purpose	Auth
/workbook-compare	POST	Runs the compare: multipart upload with workbook_id and candidate_file	Passcode
/workbook-compare/export-pdf	POST	Generates and downloads the PDF report	Passcode
/custom-views	GET	Read-only site-wide Custom View directory	Passcode
/switch-site	POST	Changes the selected Tableau site	None (see note)
/healthz	GET	Returns {"status": "ok"}	None

Note: /switch-site does not call login_required and the site switcher has no passcode check. Review this if the app is exposed beyond a trusted network.

7. Data, credentials and security

- **Tableau access** uses a Personal Access Token (PAT). The PAT secret is encrypted at rest. The PAT name is stored in plain text. The PAT needs access to every site in config.yaml.
- **Passcode** protects the local UI. It is a single shared passcode, stored only as a hash.
- **Nothing about workbooks or Custom Views is cached.** Everything is fetched live from Tableau on each request.
- **Account numbers** come from BigQuery and are cached in SQLite. They refresh once at startup, so a restart is needed to pick up new data.
- **Secrets stay out of the repo.** bigquery-credentials.json, app.log and the instance\ folder are git-ignored. Do not commit them.
- **Network exposure.** config.yaml binds to 0.0.0.0 by default, so other machines on the network can reach the app.

8. Status and known gaps

- **Live filter state (Custom View evidence)** is partly built. It is meant to capture the filter and parameter values a Custom View actually uses in the browser, via the Tableau Embedding API. The commit history shows that a previous 'Check Live State' feature was removed because the Embedding API v3 was not available. Treat this area as unfinished.
- **Impact classification** is rule-based and depends on how accurately the XML is parsed. Edge cases are expected.
- **Workbook and Custom View matching** uses the exact workbook name.
- **BigQuery sync** needs credentials. Without them, the app still runs but account numbers will be missing.
- **Documentation** is spread across IMPLEMENTATION_COMPLETE.md, LIVE_VERIFICATION_GUIDE.md, PAT_CONNECTION_GUIDE.md and PAT_QUICK_REFERENCE.md.

9. Running it locally

- Activate the virtual environment: .\.*venv\Scripts\Activate.ps1*. Use the venv, not the system Python.
- Install dependencies if needed: *pip install -r requirements.txt*.
- Start the app: .*startup.ps1*. It listens on port 5001.

- Open http://localhost:5001. First run goes to /setup. Enter the PAT name and secret, then set a passcode.
- Run tests with pytest from the project folder.

To reset the passcode, delete the *passcode_hash* row in *app_config*. The PAT is kept, and /setup asks for the PAT and a new passcode. Stop the app before doing this.

10. Glossary

Term	Meaning
Workbook (.twb / .twbx)	Tableau workbook XML. A .twbx is a packaged archive that includes the .twb and any data.
Published version	The workbook currently live on Tableau Server, downloaded for comparison.
Candidate	The proposed new version uploaded for comparison.
Custom View	A saved personal view of a workbook, owned by one user. Changing the workbook can affect it.
PAT	Personal Access Token. The credential the app uses to call Tableau's REST API.
Site	A Tableau Server site (tenant). The app works within one selected site at a time.